

Signals

Program-1:

```
himasree@LAPTOP-41COT5AA:~$ cd ~/OSLab
-bash: cd: /home/himasree/OSLab: No such file or directory
himasree@LAPTOP-41COT5AA:~$ cd ~/OSLab
himasree@LAPTOP-41COT5AA:~/OSLab$ nano signal1.c
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc signal1.c -o signal1
himasree@LAPTOP-41COT5AA:~/OSLab$ ./signal1
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
```

```
Program is running...
^C
SIGINT received!
Program is running...
```

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
void handle_signal(int sig)
{
    printf("\nSIGINT received!\n");
}
int main()
{
    signal(SIGINT, handle_signal);
    while (1)
    {
        printf("Program is running...\n");
        sleep(2);
    }
    return 0;
}
```

Program-2:

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    while (1)
    {
        printf("Program is running...\n");
        sleep(2);
    }
    return 0;
}
```

```

himasree@LAPTOP-41COT5AA:~/OSLab$ nano signal2.c
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc signal2.c -o signal2
himasree@LAPTOP-41COT5AA:~/OSLab$ ./signal2
Program is running...
Program is running...
^C
himasree@LAPTOP-41COT5AA:~/OSLab$ |

```

Program-3:

```

printf("Child is running...\n");
sleep(3);
printf("Child is terminating...\n");
}
else
{
// Parent
printf("Parent is waiting...\n");
wait(NULL);
printf("Parent finished.\n");
}
return 0;
}
|

```

```

himasree@LAPTOP-41COT5AA:~/OSLab$ nano signal3.c
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc signal3.c -o signal3
himasree@LAPTOP-41COT5AA:~/OSLab$ ./signal3
Parent is waiting...
Child is running...
Child is terminating...
SIGCHLD received: Child has terminated.
Parent finished.
himasree@LAPTOP-41COT5AA:~/OSLab$ |

```

Program-4:

```

#include <stdio.h>
#include <unistd.h>
int main()
{
int fd[2];
pipe(fd);
close(fd[0]); // Close read end
printf("Writing to pipe...\n");
write(fd[1], "Hello", 5);
printf("Write completed.\n");
return 0;
}
|

```

```

himasree@LAPTOP-41COT5AA:~/OSLab$ nano signal4.c
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc signal4.c -o signal4
himasree@LAPTOP-41COT5AA:~/OSLab$ ./signal4
Writing to pipe...
himasree@LAPTOP-41COT5AA:~/OSLab$ |

```

Practical-5:

```

#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>
void handler(int sig)
{
printf("Child received SIGUSR1\n");
}
int main()
{
{
pid_t pid = fork();
if (pid == 0)
{
// Child
signal(SIGUSR1, handler);
printf("Child waiting...\n");
while (1)
{
sleep(1);
}
}
else
{
// Parent
sleep(2);
printf("Parent sending signal...\n");

```

```

kill(pid, SIGUSR1);
sleep(2);
kill(pid, SIGTERM);
wait(NULL);
}
return 0;
}

```

```

himasree@LAPTOP-41COT5AA:~/OSLab$ nano signal5.c
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc signal5.c -o signal5
himasree@LAPTOP-41COT5AA:~/OSLab$ ./signal5
Child waiting...
Parent sending signal...
Child received SIGUSR1
himasree@LAPTOP-41COT5AA:~/OSLab$ |

```

Practical-6:

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    printf("Program started\n");
    abort();
    printf("Program ended\n");
    return 0;
}
```

```
himasree@LAPTOP-41COT5AA:~/OSLab$ nano signal6.c
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc signal6.c -o signal6
himasree@LAPTOP-41COT5AA:~/OSLab$ ./signal6
Program started
Aborted (core dumped)
```

Practical-7:

```
#include <stdio.h>
int main()
{
    int *p = NULL;
    printf("Before error\n");
    *p = 10;
    printf("After error\n");
    return 0;
}
```

```
himasree@LAPTOP-41COT5AA:~/OSLab$ nano signal7.c
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc signal7.c -o signal7
himasree@LAPTOP-41COT5AA:~/OSLab$ ./signal7
Before error
Segmentation fault (core dumped)
```